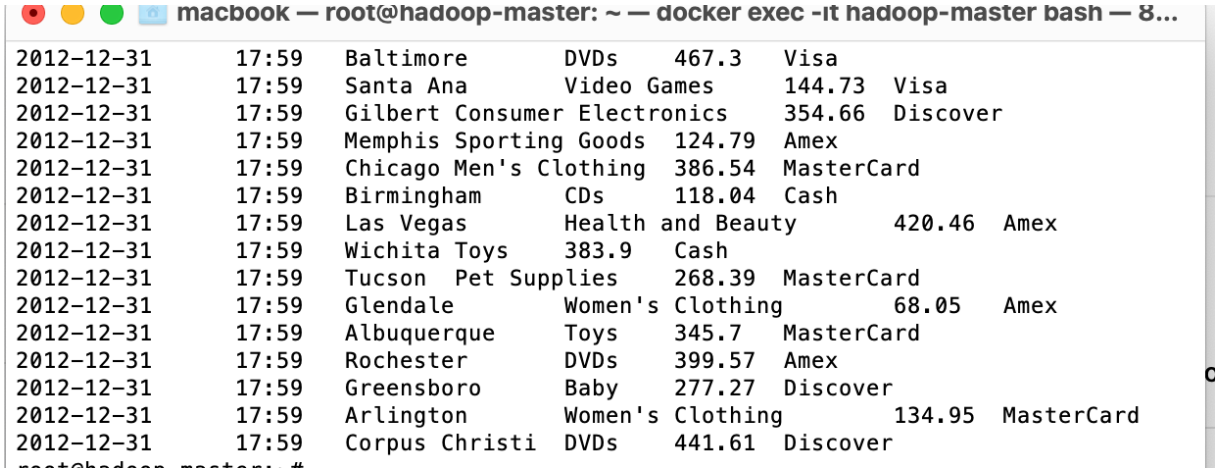
Exercice 1 – Analyse des ventes par catégorie de produit

1/Identifier la position de chaque champ dans une ligne :

hdfs dfs -cat ./input/purchases.txt



1. date
2. temp
3. magasin
4. produit
5. cout
6. paiement

2. Concevoir un Mapper qui extrait le produit et le coût de chaque transaction

(date,temp,magsin,produit,cout,paiement) ->Mapper -> (produit , coût)

3. Émettre une paire clé/valeur permettant de regrouper les ventes par produit

(clé/valeur) : (produit ,cout)

Example :

(Toys,345.7)

(DVDs,399.57)

4. Concevoir un Reducer qui calcule la somme des coûts pour chaque produit.

Grouper chaque produit et calculer la somme de cout

Example :

DVDs : 399.57 +467.3+.....=somme

5. Développer le Driver/Main et configurer correctement le Job.

Mapper code :

```java
package hadoop.mapreduce.tp2;

import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class SalesMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
    private Text product = new Text();
    private DoubleWritable cost = new DoubleWritable();

    Tabnine | Edit | Test | Explain | Document
    @Override
    protected void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
        String[] f = value.toString().split(regex: "\t");
        if (f.length != 6) return;
        try {
            product.set(f[3].trim());
            cost.set(Double.parseDouble(f[4].trim()));
            context.write(product, cost);
        } catch (NumberFormatException e) {
            // invalid line ignored
        }
    }
}
```

Reducer :

```java
package hadoop.mapreduce.tp2;

import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class SalesMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
    private Text product = new Text();
    private DoubleWritable cost = new DoubleWritable();

    Tabnine | Edit | Test | Explain | Document
    @Override
    protected void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
        String[] f = value.toString().split(regex: "\t");
        if (f.length != 6) return;
        try {
            product.set(f[3].trim());
            cost.set(Double.parseDouble(f[4].trim()));
            context.write(product, cost);
        } catch (NumberFormatException e) {
            // invalid line ignored
        }
    }
}
```

Driver :

```java
package hadoop.mapreduce.tp2;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class driver {
    Tabnine | Edit | Test | Explain | Document | Run | Debug
    public static void main(String[] args) throws Exception {
        Job job = Job.getInstance(new Configuration(), jobName: "Sales by product");
        org.apache.hadoop.fs.FileSystem fs = org.apache.hadoop.fs.FileSystem.get(job.getConfiguration());
Path out = new Path(args[1]);
if (fs.exists(out)) fs.delete(out, recursive: true);
        job.setJarByClass(cls: driver.class);
        job.setMapperClass(cls: SalesMapper.class);
        job.setCombinerClass(cls: SalesReducer.class);
        job.setReducerClass(cls: SalesReducer.class);
        job.setOutputKeyClass(theClass: Text.class);
        job.setOutputValueClass(theClass: DoubleWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(verbose: true) ? 0 : 1);
    }
}
```

7. Déposer les données dans HDFS, puis exécuter le Job sur le cluster

root@hadoop-master:~# HADOOP_CLASSPATH=/root/sales.jar hadoop hadoop.mapreduce.tp2.driver sales_input sales_output

